

Anlamların Sınırları: Wittgenstein'dan Ontolojiye ve Domain-Driven Design'a

Ömer Faruk Yavuz

July 2026

Giriş

Sorun ve Amaç

Bu çalışma tek bir soruyu izleyerek felsefeden yazılım mühendisliğine uzanan bir güzergâhı incelemektedir. Soru şudur: *aynı kelime farklı topluluklarda farklı anlamlara geldiğinde, bu farklılık tek bir bilgi sisteminde nasıl temsil edilir?*

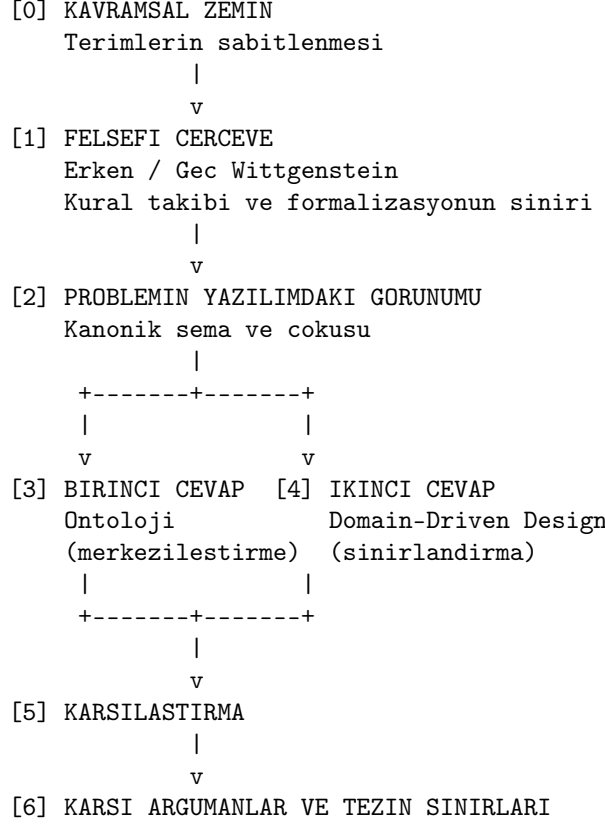
Söz konusu sorun önce felsefi bir tespit olarak ortaya çıkmış, ardından bir mühendislik problemine dönüşmüş ve nihayetinde birbirinden ayrılan iki kurumsal cevap üretmiştir. Çalışmanın amacı bu iki cevabı — Palantir Ontology ile Domain-Driven Design'ı — ortak felsefi zeminleri üzerinden karşılaştırmak ve aralarındaki farkın teknik değil kurumsal niteliğini göstermektir.

Tez

Çalışmanın savunduğu tez üç önermeden oluşmaktadır.

1. Her iki yaklaşım da aynı felsefi tespitten hareket etmektedir: kavramlar topluluğa görelidir ve tek bir kanonik şemaya sıkıştırılamaz.
2. Verdikleri cevaplar zıt yönlüdür. Domain-Driven Design farkı koruyup sınırlar arasına açık bir çeviri katmanı yerleştirir; Ontology farkı ortak bir nesne katmanında eritir.
3. Aradaki en kritik ayrım performans veya ifade gücü değil, *gücün görünür-lüğüdür*. Domain-Driven Design hangi topluluğun dilinin geçerli olacağını açık bir tasarım kararına dönüştürür; Ontology aynı kararı ortadan kaldırmaz, yalnızca altyapı konfigürasyonuna gömerek görünmez kılar.

Yol Haritası



Yöntem ve Sınırlılıklar

Çalışmanın kanıt tabanında, açıkça beyan edilmesi gereken bir asimetri bulunmaktadır.

Kaynak asimetrisi. Domain-Driven Design tarafı, öğretisini doğrudan ortaya koyan birincil ve ikincil metinlerden okunmuştur: Evans'ın kurucu kitabı, Vernon'un uygulama kitabı ve Ürkmez'in bu öğretiyi Türkçede sistematik biçimde işleyen dört yazılık serisi. Ontology tarafı ise büyük ölçüde ürün dokümantasyonundan, şirket beyanlarından ve üçüncü taraf teknik incelemelerden okunmuştur. Birinci grup *nasıl yapılması gerektiğini* anlatan normatif metinlerdir; ikinci grup ise *ürünün nasıl sunulduğunu* gösteren metinlerdir.

Bunun sonucu şudur: karşılaştırma, doğası gereği Domain-Driven Design'ı en olgun hâliyle, Ontology'yi ise beyan edilmiş hâliyle ele almaktadır. Bu eşitsizlik ortadan kaldırılamamıştır; altıncı bölümde açıkça bir karşı argüman olarak ele alınmaktadır.

Ampirik dayanağın yokluğu. Çalışma kavramsal bir analizdir. Vaka incelemesi, ölçüm veya saha gözlemi içermemektedir. Dolayısıyla ileri sürülen tez,

bir hipotez olarak okunmalı ve ampirik sınımaaya açık kabul edilmelidir.

Felsefi okumanın niteliđi. Wittgenstein'a yapılan başvuru araçsaldır, metin şerhi niteliğinde değildir. Amaç, mühendislik tartışmasına kavramsal bir çerçeve kazandırmaktır; Wittgenstein yorumculuğuna katkı sunmak değildir. Çalışma boyunca kullanılan Türkçe terimler için Aral'ın *Felsefi Soruşturmalar* çevirisi esas alınmıştır.

Kavramsal Zemin

Aşağıdaki liste hızlı başvuru amaçlıdır. Terimler ilgili bölümlerde ayrıntılı olarak ele alınmaktadır.

Anlam ve Formalizasyon

- **Anlam.** Bir işaretin neye karşılık geldiği. Bu çalışmada işaretin kendisinde bulunan bir özellik değil, kullanımdan doğan bir ilişki olarak ele alınır. *Örnek:* “Yeşil ışık” trafikte geçme izni, projede yönetici onayı, ameliyathanede hastanın stabil olduğu bilgisidir.
- **Özcülük.** Her kavramın, tüm örneklerinde ortak bulunan bir çekirdek niteliği olduğu varsayımı. *Örnek:* “Sandalye” tanımını her sıkılaştırmada bir örneği dışarıda bırakır, her gevşetmede bir yabancıyı içeri alır.
- **Dil oyunu.** Kelimenin, içine gömülü olduğu faaliyet ve o faaliyetin kurallarıyla birlikte oluşturduğu bütün. *Örnek:* Şantiyede “tuğla” bir emirdir, malzeme dersinde bir sınıflandırma birimidir, arkeolojik raporda bir tarihlendirme kanıtıdır.
- **Aile benzerliği.** Ortak tek bir nitelik bulunmamasına karşın örtüşen benzerlik zincirleriyle bir kavramın tutarlı işlemesi. *Örnek:* Maraton, satranç, dart ve otomobil yarışının hepsini kapsayan tek bir “spor” niteliği saptanamaz.
- **Formalizasyon.** Bir pratiğin açıkça yazılmış, sonlu ve makine tarafından işlenebilir kurallara dönüştürülmesi. *Örnek:* Bir tarifte malzemeler ve süreler yazılabilir; “hamur kıvamını alınca” ifadesi yazılamaz.
- **Kural takibi problemi.** Bir kuralın nasıl uygulanacağını kuralın kendisinden türetilmeyeceği tespiti. *Örnek:* “Kütüphanede sessiz olunuz” kuralı, öksürüğün ve sandalye gıcirtısının kapsam içinde olup olmadığını söylemez.
- **Örtük bilgi.** Uygulanan ancak sorulduğunda formüle edilemeyen bilgi. *Örnek:* Deneyimli kaynakçının dikişin tuttuğunu sesinden anlaması.

Modelleme ve Temsil

- **Model.** Bir alanın, belirli bir amaç doğrultusunda seçilmiş yönlerini temsil eden yapı; tanım gereği eksiktir. *Örnek:* Metro haritası coğrafi olarak yanlışdır ve işlevini tam da bu yanlışlık sayesinde görür.
- **Şema.** Verinin hangi alanlardan oluştuğunu belirleyen biçimsel tanım. *Örnek:* Formda “ikinci uyruk” kutusu yoksa o bilgi sisteme hiç girmez.
- **Kanonik.** Birden fazla aday arasından birinin resmî ve bağlayıcı sayılması. *Örnek:* Uyuşmazlıkta insan kaynaklarının yayımladığı tatil listesinin geçerli sayılması.

- **Ontoloji.** Felsefede varlığın temel kategorilerini inceleyen metafizik dalı; veri alanında bir alandaki nesne tiplerini, özelliklerini ve ilişkilerini tanımlayan yapı.
- **Ham veri.** Yorum katmanı bulunmayan kayıtlar. *Örnek:* Market fişindeki barkod ve tutar, ne alındığını söylemez.
- **Anlam katmanı.** Ham verinin gerçek dünyadaki karşılığını tanımlayan, veriyi değiştirmeyip okunuşunu belirleyen yapı.
- **Uygulamalı soyutlama.** Kâğıt üzerinde kalmayıp çalışan bir sisteme dönüşen ve bu nedenle sınanabilen soyutlama. *Örnek:* Yanlış konumlandırılmış bir kapı, planda görünmez; inşaatta ilk gün kendini gösterir.
- **Dijital ikiz.** Bir kurumun yazılımda birebir karşılığının kurulduğu iddiası. Harita ölçekli sadeleştirmedir ve sadeleştirdiğini beyan eder; ikiz eksiksizlik iddiası taşır.

Topluluk, Sınır ve Güç

- **Topluluk.** Ortak bir işi ortak kurallarla yürüten ve bu nedenle ortak bir kelime dağarcığı geliştiren grup. *Örnek:* Boş bir yatak hemşire için kapasite, muhasebe için gelir kaybıdır.
- **Bağlam.** Bir kavramın belirli bir anlama sabitlendiği alan.
- **Sınır.** İki bağlamın ayrıldığı ve kavramların anlam değiştirdiği hat. Teknik değil dilseldir. *Örnek:* Pasaport kontrol hattının asıl işlevi konum değil, kural değişimidir.
- **Çeviri ve çeviri kaybı.** Bir bağlamın kavramlarının başka bir bağlama dönüştürülmesi ve bu dönüşümde bazı ayırt edici unsurların düşmesi. *Örnek:* “Acil” kaydın sırasıyla “öncelikli” ve “bugün içinde” hâline gelmesi.
- **Asimetri.** Etkinin tek yönlü akması. Değer sıralaması değil, bağımlılık yönüdür. *Örnek:* Ana yüklenicinin kararı taşeronun planını yeniden kurar; tersi olmaz.
- **Müzakere.** Hangi bağlamın kavramının geçerli olacağının açıkça tartışılıp karara bağlanması. *Örnek:* Ortak alan temizliğinin toplantıda karara bağlanması ile bir dairenin sessizce üstlenmesi aynı sonucu üretir; ikincisi tartışılmaz, çünkü hiç kurulmamış gibidir.
- **Sessiz hata.** Sistemin çökmediği, uyarı üretmediği, ancak yanlış çalıştığı durum. *Örnek:* Sabit biçimde şaşan bir terazi, her tartımda tutarlı ve yanlış sonuç verir.

Domain-Driven Design Terimleri

Türkçe karşılıkları yerleşmediği için özgün biçimleriyle bırakılmıştır.

- **Domain.** Organizasyonun varlık nedeni ve bilgisinin bütünü. Banka için bankacılık, havayolu için havacılık.
- **Problem space / Solution space.** Hangi problemin önemli olduğu sorusunun alanı ile bunun yazılımda nasıl karşılanacağı sorusunun alanı.
- **Subdomain.** Domain'in mantıksal parçası; Core, Supporting veya Generic olarak sınıflandırılır. Problem space'e aittir.
- **Bounded Context.** Bir domain modelinin yaşadığı ve kavramların tek anlama sabitlendiği sınır. Solution space'e aittir.
- **Ubiquitous Language.** Bir bağlam içinde domain uzmanları ile geliştiricilerin birlikte kurduğu ve toplantıda, dokümantasyonda ve kodda aynı biçimde kullanılan dil.
- **Domain expert.** İşi herkesten iyi bilen kişi; unvan değil konum.
- **Entity / Value Object.** Kimliği olan ve zaman içinde değişen nesne ile kimliği olmayıp yalnızca değerleriyle tanımlanan nesne.
- **Aggregate.** Tutarlılığından tek bir kök nesnenin sorumlu olduğu nesne kümesi.
- **Context Map.** Bağlamların ve aralarındaki ilişkilerin haritası; hedeflenen mimariyi değil mevcut durumu çizer.
- **Anemic Domain Model.** Veri taşıyıp davranış taşımayan modellerin oluşturduğu anti-pattern.
- **Big Ball of Mud.** Sınırların tutarsızlaştığı ve kavramsal düzenin kaybedildiği sistem bölgesi.

Terminoloji Notu

Küçük harfle “ontoloji” genel kavramı, “Ontology” Palantir ürününü belirtmektedir. Benzer biçimde “bağlam” genel anlamda, “Bounded Context” Domain-Driven Design'in teknik terimi olarak kullanılmaktadır. Bu ayrım çalışmanın konusunun doğrudan bir gereğidir: aynı kelimenin iki dil oyununda iki farklı nesneye karşılık gelmesi, incelenen olgunun kendisidir.

Felsefi Çerçeve

Ontoloji Teriminin İki Yaşamı

Ontoloji terimi iki ayrı alanda, iki farklı anlamda kullanılmaktadır ve aralarındaki bağ türeyimseldir.

Felsefi kullanım

Felsefede ontoloji, “ne vardır?” sorusunu ele alan metafizik daldır. Varlığın en temel kategorilerini inceler: dünya nihai olarak nelerden oluşmaktadır, hangi şeyler gerçekten vardır ve hangileri tüevdir? Nesneler mi vardır yoksa süreçler mi? Bir orman var mıdır, yoksa yalnızca ağaçlar mı vardır ve orman bizim koyduğumuz bir etiket midir?

Felsefi ontoloji, kısaca, gerçekliğin envanterini ve o envanterin kategorilere ayrılma biçimini sorgular.

Teknik kullanım

Veri alanındaki anlam daha somuttur: bir ontoloji, belirli bir alandaki nesne tiplerini, bunların özelliklerini ve aralarındaki ilişkileri açıkça tanımlayan yapıdır.

Ham veri, tablo düzeyinde birbirinden kopuk hücrelerden oluşur. Ontoloji ise bu hücrelerin üzerine şu iddiayı yerleştirir: burada gerçekte bir **Kayıt** nesnesi, bir **Kullanıcı** nesnesi ve bir **İndirme** olayı bulunmaktadır; ve bunlar arasında tanımlı ilişkiler vardır.

Bu haliyle ontoloji, “satır ve sütun” temsilinden “nesne ve ilişki” temsiline geçmiştir: verinin, makinenin depolama biçimiyle değil gerçek dünyadaki karşılığıyla modellenmesi girişimidir.

Erken Wittgenstein: Resim Kuramı

Tractatus Logico-Philosophicus’un konumu şöyle özetlenebilir: dil dünyanın bir resmidir, her anlamlı önerme bir olgu durumuna karşılık gelir ve doğru analiz edildiğinde tek bir mantıksal iskelet ortaya çıkar. Dünya, şeylerin değil olguların toplamıdır.

Bu konum formal, sonlu ve tam bir temsil idealini savunur. Modern veri modellemesinin örtük felsefesi büyük ölçüde budur; bu tespit çalışmanın ilerleyen bölümlerinde belirleyici olacaktır.

Geç Wittgenstein: Dil Oyunları ve Aile Benzerliği

Felsefi Soruşturmalar, bu konumun yazarı tarafından terk edildiği metindir. Merkezî iddia şudur:

Bir kelimenin anlamı, dildeki kullanımıdır.

Bu iddiaya göre bir kavramın çekirdek tanımı yoktur. Kavramı anlamlı kılan şey, belirli bir topluluğun onu belirli pratikler içinde nasıl kullandığıdır: kimi kapsadığı, kimi dışladığı ve hangi eylemi tetiklediği. Wittgenstein bu bütünlüğe **dil oyunu** adını verir; kelime, kurallarıyla birlikte bir faaliyetin içine gömülüdür. Faaliyet değiştiğinde anlam da değişir, çünkü anlam zaten faaliyetin içinde bulunmaktaydı.

İkinci kavram **aile benzerliğidir**. “Oyun” kelimesinin kapsadığı örneklerin tümünde bulunan tek bir özellik saptanamaz. Satranç, futbol, solitaire ve çocukların yakalamacası arasında kimisi rekabetçidir kimisi değildir; kimisi katı kurallıdır kimisi gevşektir. Ortak bir öz yoktur; örtüşen benzerlik zincirleri vardır. Bu tespit, “yeterince kesin bir tanım yapılırsa kavram yakalanır” varsayımına doğrudan bir itirazdır.

Kural Takibi ve Formalizasyonun Sınırı

Geç Wittgenstein’in en radikal iddiası çoğunlukla atlanan bir noktadadır. İddia yalnızca “anlam kullanımdadır” değil, “anlam tam olarak formalize edilemez”dir.

Kural takibi argümanının vardığı sonuç şudur: hiçbir kural kendi uygulamasını belirlemez. Bir kuralın nasıl uygulanacağı, kuralın kendisinden türetilemez; her formalizasyon, kendisinin nasıl okunacağını belirleyemeyen bir pratiğe yaslanır.

Bu sonuç, çalışmanın ilerleyen bölümlerinde ele alınacak gerilimin kaynağıdır: incelenen mühendislik cevaplarının her ikisi de, tam olarak anlamı formalize etme girişimidir. Dolayısıyla ikisi de aynı felsefi itirazın hedefindedir ve aralarındaki fark, itirazı ne ölçüde karşıladıklarında değil, itiraza rağmen neyi feda ettiklerinde aranmalıdır.

Problemin Yazılımdaki Görünümü

Aynı Kelime, Farklı Nesne

Bir bankada “müşteri” kelimesinin dört ayrı birimdeki kullanımı incelendiğinde şu tablo ortaya çıkmaktadır:

- **Satış** için müşteri, potansiyel gelirdir; henüz hesabı olmayan biri de müşteridir.
- **Risk** için müşteri, yükümlülük taşıyan taraftır; vefat etmiş biri, kredisi devam ettiği sürece hâlâ müşteridir.
- **Hukuk** için müşteri, imzalı sözleşmesi olan tüzel kişidir; şirketin sahibi müşteri değildir, şirket müşteridir.
- **Destek** için müşteri, arayan kişidir; sözleşme sahibinin eşi de müşteridir.

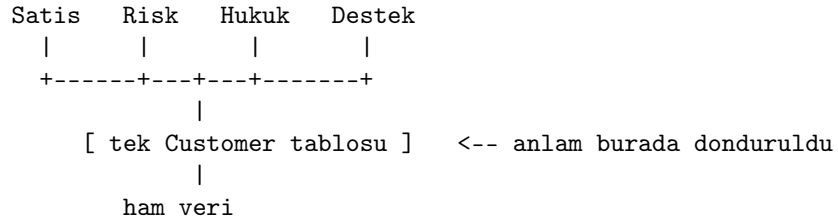
Bunlar aynı kelimenin farklı yorumları değil, **farklı nesnelerdir**. Her birinin işi, “müşteri”nin sınırlarını farklı bir yerden çizmeyi zorunlu kılmaktadır.

Aynı yapı savunma ve güvenlik alanında “hedef” kavramında da gözlenir. İstihbarat analisti için hedef, gözetlenen bir varlıktır; zamanla değişen konumu, bilinen ortakları ve güven skoru olan bir şeydir ve hedef olmak geri alınabilir bir statüdür. Dolandırıcılık analisti için hedef ise bir işlem örüntüsüdür; kişi olmak zorunda değildir, bir kart veya bir cihaz tanımlayıcısı olabilir ve hedef olmak bir skor eşiğinin aşılmasıdır.

Kanonik Şema ve Çöküşü

Klasik veri mühendisliği yaklaşımı, felsefi konumu itibarıyla erken Wittgenstein’cidir: kurumda tek bir doğru **Customer** tablosu bulunur, tüm birimler ona uyar ve uymayan kendi tarafında çeviri yapar.

KANONİK SEMA MODELİ



Yapısal sorun şudur: söz konusu tablo, taraflardan birinin dil oyununa göre yazılmıştır. Diğerleri kendi kavramlarını bu şemaya çevirmek zorunda kalır ve çeviride belirleyici olan unsur kaybolur. Risk birimi vefat etmiş müşteriyi, destek birimi sözleşme sahibinin eşini kaybeder.

Kaybın niteliği belirleyicidir. Kayıp gürültülü değil sessizdir: sistem çökmez, yalnızca yanlış çalışır. Bu nedenle teknik izleme araçlarıyla saptanamaz. Gözlemlenebilir sonucu, birimlerin kendi kayıt dışı tablolarını tutmaya başlamasıdır.

Ara Tespit: Çevirinin Kaçınılmazlığı

Buradan çıkan ve her iki cevabın da örtük olarak kabul ettiği ilke şudur:

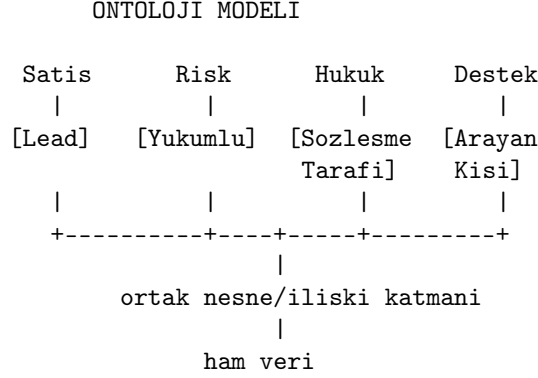
Çeviri yok edilemez; yalnızca nerede yapılacağı seçilebilir.

Kötü tasarımda çeviri her yerdedir ve görünmezdir. İyi tasarımda çeviri sınırlı sayıda noktada toplanmıştır ve bilinçlidir. İncelenen iki yaklaşım, bu noktaların nerede olacağı konusunda ayrışmaktadır.

Birinci Cevap: Ontoloji

Yapı

Ontoloji yaklaşımı kanonik şemayı tersine çevirir:



İlke şudur: ham veri tektir, anlam katmanı çoğuldur. Her topluluk kendi nesne tiplerini, özelliklerini ve ilişkilerini tanımlar; bu tanımlar aynı kayıtların üzerine oturur ancak farklı kesitler alır.

Üreticinin kendi tarifiyle ontoloji, bir veri kataloğu veya şema tasarımı çözümü değil kurumun *operasyonel anlam katmanıdır*: nesneler, özellikler ve bağlantılardan oluşan semantik unsurlar ile bunlar üzerinde çalışan eylem ve fonksiyonlardan oluşan kinetik unsurları birlikte barındırır.

Ürünün çekirdek fikri, analistin zihinsel modelini yazılımda doğrudan temsil etmektir. Bir istihbarat analisti “şu numara şu kişiye ait, o kişi şu araca binmiş, o araç şu adreste görülmüş” biçiminde, nesneler ve ilişkiler üzerinden düşünür; oysa veri birbirinden kopuk veritabanlarında tablolar hâlinde durur. Ontoloji katmanı bu iki temsil arasındaki mesafeyi kapatır ve kullanıcının sorgu dili yerine ilişki grafiği üzerinde gezinmesini mümkün kılar.

Mimari Kısıtlar

Ürünün üç mimari özelliği, çalışmanın karşılaştırma bölümü açısından belirleyicidir.

Kapsam zorunluluğu. Veri yerinde sorgulanmaz; işleme hatlarından geçirilerek platforma alınır ve ontoloji nesnelere dönüştürülür. Bu, kısmi uygulamayı zorlaştırır: ontolojinin değeri bağlantı kurabilmesinden geldiği için, kapsam dışında kalan her alan aynı zamanda değer kaybıdır.

Ontoloji-alan eşleşmesi. Bir ontoloji, platformdaki bir çalışma alanıyla birebir eşleşir. Bu, ilerleyen bölümde ele alınacağı üzere, bir sınır kavramının tümüyle yok olmadığını ancak farklı bir ölçekte konumlandığını göstermektedir.

Ontolojiler arası bağ yasağı. Farklı ontolojilerdeki varlıklar birbirine bağlanamaz. Bu kısıt, çoklu ontolojiyi teknik olarak mümkün kılmakla birlikte pra-

tikte cazibesiz hâle getirir, zira ürünün temel vaadi olan bağlantılanabilirlik tam da bu sınırdan kesilir.

Ontoloji Uygulamalı Soyutlamadır

Ontoloji kurmak uygulamalı bir soyutlama işlemidir. Yüzeydeki değişken ayrımı — hangi tablo, hangi sütun, hangi format — atılır; altındaki yapı tutulur.

Kavramsal soyutlamalardan farkı sınanabilirliğidir. Kâğıt üzerinde kalan bir kavramsal iskeletin yanlışlığı uzun süre görünmez kalabilir. Ontoloji ise çalışan bir şemaya dönüşür ve yanlış kurulduğunda sistem işlevsizleşir. Bu anlamda ontoloji, gerçeklikle çarpışarak sertleşmiş bir metafiziktir.

Bu özellik, çalışmanın Ontology'ye yönelttiği eleştirilerden bağımsız olarak, yaklaşımın kayda değer bir epistemolojik erdemidir.

Erken mi, Geç mi Wittgenstein?

Ontology'yi geç Wittgenstein'in ürüne dönüşmüş hâli olarak sunan yaygın anlatı tam oturmamaktadır ve bu uyumsuzluk kavramın anlaşılmasını derinleştirmektedir.

Yapı erken Wittgenstein'a benzemektedir. Nesneler, özellikler ve ilişkilerden kurulu formal, sonlu ve makine tarafından işlenebilir bir model, doğrudan *Tractatus*'un dünya tasviridir.

Gerekçe geç Wittgenstein'a benzemektedir. Neden tek şema değil de çoğul şema sorusuna verilen cevap, anlamın topluluğa göreliliğine dayanır.

Yani ürün, çoğulculuğun *neden* gerektiğini geç Wittgenstein'dan, *nasıl* modelleneyeceğini erken Wittgenstein'dan almaktadır. Burada gerçek bir felsefi ironi bulunmaktadır: geç Wittgenstein'ın asıl iddiası anlamın tam olarak formalize edilemeyeceğidir; ontoloji ise tam olarak anlamı formalize etme girişimidir.

Dürüst formülasyon şudur: ontoloji, Wittgenstein'ın tespitini ciddiye alır ancak uyarısını göz ardı eder. Bu bir kusur değildir; mühendislik zaten böyle çalışır. Ancak "Wittgenstein'ın ürüne dönüşmüş hâli" ifadesi, olduğundan daha temiz bir düşünsel devamlılık ima etmektedir.

Olgusal Düzeltme

Yaygın anlatıda düzeltilmesi gereken bir olgusal hata bulunmaktadır: Palantir'in ilk ürününün adı Wittgenstein'a atıf değildir. Şirket adı Tolkien'den gelmektedir (*palantír*, gören taş); ilk ürün Palantir Government, sonrasında Gotham'dır. Wittgenstein bağlantısı ürün adında değil, ontoloji kavramının kendisindedir ve o kelimenin kökü Aristoteles'e kadar gitmekte olup Wittgenstein'a özgü değildir.

Alex Karp'ın Frankfurt'ta tamamladığı doktora tezinin başlığı *Aggression in der Lebenswelt*'tir. Bu tezin "ontoloji" terimine başvurmayı ideolojik bir işlem olarak eleştirdiği yönündeki iddia ikincil kaynaklarda dolaşmaktadır; ancak birincil metin üzerinden doğrulanmamıştır ve bu çalışmada kanıt olarak kullanılmamaktadır.

Arşivlenecek doğru formülasyon şudur: ontoloji kavramının savunusu geç Wittgenstein'cı bir argümana dayanmakta, yapısı ise erken Wittgenstein'cı kalmaktadır.

İkinci Cevap: Domain-Driven Design

Bu bölüm, yaklaşımın yalnızca karşılaştırma için gerekli unsurlarını özetlemektedir. Ayrıntılı sunum için kaynakçadaki metinlere başvurulmalıdır.

Değer Önerisi ve Uygulanabilirlik Koşulu

Yaklaşımın tek cümlelik değer önerisi, domain bilgisini merkezileştirmek ve korumaktır. Teknik detayların, veri modelinin, arayüz ihtiyaçlarının ve çerçeve alışkanlıklarının iş bilgisine sızıp onu bozması engellenir.

Yaklaşımın kendisi bir maliyet kalemidir; başlangıç yatırımı yüksektir ve ilerleme başlangıçta yavaş hissedilir. Bu nedenle uygulanabilirlik koşulu açıkça tanımlanmıştır: kolayca değiştirilebilen sistemlerde uygulanmaz, uzun ömürlü ve iş açısından kritik sistemler içindir. Belirleyici soru karmaşıklığın kaynağıdır — problem doğası gereği mi zordur, yoksa çözüm mü karmaşılaştırılmaktadır?

Stratejik ve Taktiksel Ayrım

	Strategic Design	Tactical Design
Soru	Neyi, neden inşa ediyoruz?	Bunu nasıl modelleriz?
Araçlar	Domain, Subdomain, Bounded Context, Context Map	Entity, Value Object, Aggregate, Domain Service
Odak	İş ve organizasyon yapısı	Kod

Kritik bağımlılık şudur: taktiksel tasarım, stratejik tasarımın çizdiği sınırlar içinde anlam kazanır ve sıra tersine çevrilemez. Stratejik tasarımın kapsamı yalnızca yazılımla sınırlı değildir; ekipler arası sınırları da belirlediği için organizasyonu da tasarlar.

DDD Lite

Stratejik kısmın atlanıp yalnızca taktiksel araçlara odaklanıldığı uygulama biçimi literatürde DDD Lite olarak adlandırılmaktadır. Dışarıda bırakılan üç unsur şunlardır: ortak dil inşası, bağlamsal sınırlar ve bağlamlar arası ilişki haritası. Bunların yerine sırasıyla teknik terminoloji, kod içi klasörleme yapısı ve tasarlanmamış entegrasyonlar geçer.

Bu kavram, çalışmanın karşılaştırma bölümünde merkezî bir işlev görecektir.

Bounded Context: Dilsel Sınır

Bounded Context, bir domain modelinin yaşadığı sınırdır ve sınır içinde konuşulan dil Ubiquitous Language'dir. Tanımın en işlevsel sonucu bir teşhis kriteri sunmasıdır:

Bir kavramın birden fazla anlama geldiği düşünülüyorsa, bağlam fazla büyümüştür; aslında birden fazla bağlamın içinde bulunulmaktadır.

Kanonik örnek, bankacılık domain’indeki **Account** kavramıdır. Vadesiz hesap bağlamında **Account**; para yatırma, çekme, bakiye takibi ve günlük işlem limitleri davranışlarına sahiptir. Vadeli hesap bağlamında ise faiz hesaplama, vadeye bağlı kurallar ve birikim davranışlarıyla tanımlanır. Her iki bağlamda da nesnenin adı **Account** olabilir; anlam farkını sınır belirlemektedir.

Sınırın kapsamı yalnızca domain katmanı değildir: kalıcılık şeması, uygulama servisleri, arayüz katmanı ve dışa açılan uç noktalar da bu sınırın içindedir. Sahiplik ilkesi “tek bağlam, tek dil, tek takım” biçiminde formüle edilir.

Ubiquitous Language

Ubiquitous Language, bir işi anlatmak için kullanılan kelimelerin, o işin geçtiği her yerde aynı tutulması disiplini. Gerekçesi çeviri maliyetidir: bir kavram aktarım zinciri boyunca farklı isimler aldığı anda, her aktarımda taşıdığı bir unsur sessizce düşer.

Yaklaşımın üç olumsuz tanımı belirleyicidir. Ubiquitous Language *yalnızca iş tarafının dili değildir*, zira iş tarafı kendi içinde tutarlı olmak zorunda kalmamıştır. *Sektör terminolojisi değildir*, zira genel terimler kurumu ayırt eden farklılığı adsız bırakır. *Domain uzmanının tek başına dili de değildir*, zira işi en iyi bilen kişi onu anlatmakta en iyi kişi değildir; uzun süredir bir işi yapan kişi kuralları refleks olarak uygular ve sorulduğunda formüle edemez.

Dil, domain uzmanı ile geliştiricinin temas noktasında doğar: birincisi bilgiyi, ikincisi kesinlik talebini getirir.

Sürtünme sinyali

Epistemolojik olarak en değerli tespit şudur: bir kelimeyi kullanırken duyulan rahatsızlık genellikle kelime sorunu değil, anlayış sorununun göstergesidir. “Sipariş” kelimesi rahat kullanılırken “sepete eklendi ama ödenmedi, bu da sipariş midir?” sorusu karşısında duraksanıyorsa, kelimenin iki farklı şeyi birden taşıdığı ortaya çıkmıştır.

Ters durumda ise şu gözlenir: hiç rahatsızlık üretmeyen kelimelerin ortak özelliği hiçbir şey söylememeleridir — “kayıt”, “bilgi”, “işlem”, “birim”, “öge”. Bunlar hiçbir yere yerleştirilirken zorlanma yaratmaz, çünkü hiçbir şeyi ayırt etmezler. Sıfır sürtünme genellikle sıfır anlam göstergesidir.

Anemic Domain Model

Fowler’ın 2003’te adlandırdığı bu anti-pattern, veri taşıyıp davranış taşımayan modelleri tanımlar.

Kavramı somutlaştırmak için şu karşıtlık kullanılabilir. Bir varlık iki biçimde temsil edilebilir: birincisinde varlık kendi kilidine sahip bir kasadır ve talep edilen işlemi gerçekleştirmeden önce koşulları kendisi denetler; ikincisinde varlık üzerinde bir değer yazılı bir kâğıttır ve kuralları bilen ayrı birimler bulunur. Anemic Domain Model ikinci temsildir.

Asıl maliyet, kuralların model dışına dağılmasının iki sonucudur. Birincisi, yeni eklenen bir yolun mevcut kuralı farkında olmadan atlayabilmesidir; model itiraz etmediği için hata sessizce geçer. İkincisi, bir kural değiştiğinde onu bilen tüm birimlerin tek tek bulunup düzeltilmesi gereğidir; biri atlandığında kural sistemde iki farklı biçimde işlemeye başlar.

Nüans önemlidir: anemik model her zaman hata değildir ve korunacak bir kuralın bulunmadığı basit alanlarda bilinçli bir tercih olabilir. Asıl sorun, anemik model kullanıldığının fark edilmemesidir.

Context Map ve İlişki Pattern'leri

Context Map, bağlamların ve aralarındaki ilişkilerin haritasıdır. Belirleyici ilkesi bugünü çizmesidir: hayal edilen mimariyi değil mevcut durumu gösterir.

Harita üç katmanı aynı anda temsil eder: modeller arasındaki ilişki, dillerin sınırdaki nasıl uyumlanacağı ve ekipler arasındaki organizasyonel dinamik. Üçüncü katman, başka hiçbir diyagram türünde bulunmaz ve bu çalışmanın karşılaştırma bölümünün dayanak noktasıdır.

Dokuz pattern

Pattern	Özü	Bedeli
Partnership	Ortak kader, ortak sürüm	Sürekli koordinasyon
Shared Kernel	Modelin bir parçası ortak	Çok yakın bağımlılık
Customer/Supplier	Talepler planlamaya girer	Pazarlık maliyeti
Conformist	Üst model aynen benimsenir	Kontrolün devri
Anticorruption Layer	Sınırdaki çeviri katmanı	Katmanı yaşatma
Open Host Service	Çok tüketiciye açık protokol	Protokol taahhüdü
Published Language	Belgeli ortak yayın dili	Sürüm yönetimi
Separate Ways	Bilinçli olarak bağlanmama	İşlevin yinelenmesi
Big Ball of Mud	Karmaşayı karantinaya alma	Kurtarmadan vazgeçme

Bu dokuz pattern, esasen tek bir soruya verilen farklı cevaplardır: iki topluluk anlaşamadığında kimin dili geçerli olacaktır?

İki uyarı, karşılaştırma bölümü açısından özellikle önemlidir.

İlan edilmemiş Shared Kernel. Bir bağlaman veritabanı diğerinin tarafına çoğaltıldığında, resmen kararlaştırılmadan bir Shared Kernel yaratılmış olur. Çoğaltma, beklenenin aksine özerklik getirmez; iki tarafı aynı şemaya bağlar ve bu bağ hiç müzakere edilmemiştir.

Bilinçsiz Conformist. Conformist olmak bilinçli yapıldığında makul bir ödünleşimdir; asıl sorun farkında olmadan Conformist olmaktır. Dış bir arayüzün veri taşıyıcıları domain modelin içine kadar sızmışsa, karar verilmeden verilmiş demektir.

Son olarak, roller bağlama değil ilişkiye aittir: aynı bağlam bir ilişkide üst konumda, diğerinde alt konumda bulunabilir.

Karşılaştırma

Kavram Eşlemesi

DDD	Ontology	Durum
Entity / Value Object	Object Type / Property	Yakın
Domain Service	Function	Yakın
Domain Event	Action	Kısmi
Ubiquitous Language	Nesne tipi isimlendirmesi	Kısmi
Bounded Context	Ontoloji-alan eşlemesi	Farklı ölçekte
Aggregate	—	Karşılığı yok
Context Map	—	Karşılığı yok
İlişki pattern'leri	—	Karşılığı yok
Subdomain tipolojisi	—	Karşılığı yok

Tablodaki en önemli satır Bounded Context satırındır ve dikkatli okunmalıdır. Ontology'de bir sınır kavramının bulunmadığını söylemek yanlış olur: ontolojiler çalışma alanlarıyla eşleşmekte ve aralarında bağ kurulamamaktadır. Doğru tespit, sınırın *yok olması* değil *ölçek değiştirmesidir*. Domain-Driven Design'da sınır kurumun içinden geçer ve topluluklar arasına konur; Ontology'de sınır kurumun etrafını çevirir ve dış dünyaya karşı konur.

Bu ayrımın sonucu şudur: karşılığı bulunmayan tüm kelimeler stratejik taraftadır ve Ontology, Domain-Driven Design'ın taktiksel yarısıyla önemli ölçüde örtüşmekte, stratejik yarısıyla örtüşmemektedir.

Birinci Ayrım: Gücün Görünürlüğü

Context Map'in dokuz pattern'i bir müzakere spektrumu oluşturur:

KIMIN DILI GECERLİ?

Partnership	-> ikisi birlikte karar verir
Shared Kernel	-> ortak parça, ortak sahiplik
Customer/Supplier	-> üst taraf karar verir, alt taraf pazarlık eder
Conformist	-> üst tarafın dili aynen kabul edilir
ACL	-> alt taraf kendi dilini korur, sınırdan çevirir
OHS/PL	-> üst taraf herkese tek bir dil yayınlar
Separate Ways	-> hic konuşmama kararı
Big Ball of Mud	-> dil kaybedilmiş, karantina

Ontology'de bu spektrum bulunmaz; tek bir konfigürasyon vardır ve tüm birimler ona bağlanır. Domain-Driven Design terimleriyle ifade edilirse Ontology, kurum çapında ilan edilmiş bir Shared Kernel ve o kernel'e karşı herkesin Conformist olmasıdır.

İki uyarı burada birleşmektedir. Shared Kernel için kural, paylaşılan parçanın olabildiğince küçük tutulmasıdır; Ontology ise paylaşılan parçayı bilinçli olarak büyütür, zira değeri kapsamından gelmektedir. Conformist için kural,

kararın bilinçli alınmasıdır; Ontology ise bu kararı bir konfigürasyon işlemine dönüştürerek görünmezleştirir.

Sonuç şudur: Ontology, gücün nerede olduğu sorusunu ortadan kaldırmaz; cevabı altyapıya gömer. Hangi topluluğun kavram dünyasının kanonik hâle geldiği bir tasarım kararıdır, ancak hiçbir yerde bu adla kaydedilmez.

İkinci Ayrım: Seçicilik

Core, Supporting ve Generic ayrımı bir sınıflandırma egzersizi değil bir yatırım kararıdır; amaç her alanı iyileştirmek değil, değer üreten bilgiyi korumaktır.

Ontology’de bu ayrım bulunmaz. Ontology kapsayıcıdır; değeri birbirinden uzak veriler arasında bağlantı kurabilmesinden gelir ve bu değer ancak yüksek kapsama oranıyla gerçekleşir.

İki farklı bahis söz konusudur:

- **Domain-Driven Design’in bahsi:** Karmaşıklık her yerde eşit değildir; çaba Core’a yoğunlaştırılmalıdır.
- **Ontology’nin bahsi:** Asıl değer bağlantıdadır; bu nedenle her şey kapsam içinde olmalıdır.

Pratik sonucu, Ontology’nin kısmi uygulanmaya elverişsiz olmasıdır. Domain-Driven Design ise seçici uygulanabilir; yalnızca Core’da uygulanıp geri kalanın basit tutulması meşru bir stratejidir.

Üçüncü Ayrım: Anlamanın Yaşadığı Yer

DDD’DE ANLAM NEREDE	ONTOLOGY’DE ANLAM NEREDE
kod -> tip sistemi	konfigürasyon -> arayüz
derleme zamani kontrol	calisma zamani yorumlama
surum kontrolu, inceleme	yonetim paneli, yetki
sahibi: gelistirici ekibi	sahibi: platform/analist

Domain-Driven Design’da bir kavramın değiştirilmesi bir kod değişikliğidir: tartışılır, incelenir, geçmiş kayıtlar altına alınır ve tip sistemi tarafından denetlenir. Ontology’de bir kavramın değiştirilmesi bir konfigürasyon değişikliğidir: hızlıdır ve teknik olmayan personel tarafından yapılabilir, ancak daha az sürtünmeli ve daha az izlenebilirdir.

Bu fark, “anlamı kim korumaktadır?” sorusunun cevabını değiştirir. Domain-Driven Design’in ana metaforu duvardır — sınır, koruma, çeviri. Ontology’nin ana metaforu katmandır — serilen, her yere uzanan. Birincisinin vaadi koruma, ikincisinininki erişimdir.

Dördüncü Ayrım: Harita ile İkiz

Context Map'in tanımlayıcı ilkesi bugünü çizmesidir; bu nedenle bozulmuş bölgeler bile haritaya dâhil edilir. Ontology ise kendisini kurumun dijital ikizi olarak tanımlar.

İkiz, harita değildir. Harita ölçekli bir sadeleştirme ve sadeleştirdiğini beyan eder; ikiz eksiksizlik iddiası taşır.

Bu ayrımın yalnızca retorik olmadığı, mimari kısıtlardan çıkarılabilir. Ontology'nin kapsam zorunluluğu — kısmi uygulamanın değer üretmemesi — fiilî bir birebirlik baskısı yaratmaktadır. Yani eksiksizlik iddiası bir slogandan ibaret değil, ürünün değer önerisinin yapısal sonucudur.

Bu tespit, birinci bölümdeki felsefi gerilimi yeniden üretmektedir:

Tractatus	:	dunya <-> dil, birebir	~	dijital ikiz
Sorusturmalar	:	ortusen kısmi pratikler	~	context map

Domain-Driven Design, felsefi bir iddiada bulunmaksızın geç Wittgenstein tarafında konumlanmaktadır. Ontology ise retorik olarak geç Wittgenstein'a yaslanırken pratikte erken Wittgenstein projesini yürütmektedir.

Karşı Argümanlar ve Tezin Sınırları

Bu bölüm, çalışmanın tezine yöneltilebilecek en güçlü itirazları ele almaktadır. İtirazların bir kısmı tezi zayıflatmakta, bir kısmı ise yalnızca kapsamını daraltmaktadır.

Birinci İtiraz: Kaynak Asimetrisi

İtiraz. Çalışma, Domain-Driven Design’ı öğretisiyle, Ontology’yi ise ürün sunumuyla karşılaştırmaktadır. Adil olan, ya iki öğretiyi ya da iki pratiği karşılaştırmaktır. Mevcut kurguda ilk yaklaşım en olgun hâliyle, ikincisi ise broşür hâliyle temsil edilmektedir.

Değerlendirme. İtiraz büyük ölçüde haklıdır ve yöntem bölümünde açıkça kabul edilmiştir. Kısmi bir savunma şudur: bir ürünün mimari kısıtları — kapsam zorunluluğu, ontolojiler arası bağ yasağı — pazarlama metni değil teknik gerçektir ve çalışmanın en güçlü çıkarımları bu kısıtlara dayanmaktadır. Buna karşın “gücün görünürlüğü” argümanının pratikte nasıl işlediği, saha gözlemi olmaksızın kesin biçimde savunulamaz.

Tezin bu itiraz karşısındaki doğru statüsü şudur: sınanmayı bekleyen bir hipotez.

İkinci İtiraz: Kurumsallaşmış Ubiquitous Language

İtiraz. Çalışma, Ontology’de ortak dil inşasının bulunmadığını ima etmektedir. Oysa Palantir’in kurumsal pratiği — mühendisin uzun süre müşterinin yanında çalışması — Ubiquitous Language’in kurumsallaşmış bir biçimidir ve çoğu “DDD uyguluyoruz” beyanından daha fazla domain uzmanı teması içermektedir.

Değerlendirme. İtiraz güçlüdür ve çalışmanın kavram eşlemesi tablosunda “kısmi” olarak işaretlenen satırı destekler. Ancak tezi çürütmez, çünkü tez ortak dilin *kurulmadığını* değil, kurulan dilin *sahipliğinin* farklı olduğunu iddia etmektedir. Domain-Driven Design’da dil, sınır içindeki ekibe aittir ve o ekibin kod tabanında yaşar. Ontology’de dil, platformu yapılandıran tarafa aittir ve konfigürasyonda yaşar. Yoğun domain teması, bu sahiplik farkını ortadan kaldırmaz.

Üçüncü İtiraz: Çoklu Ontoloji İmkânı

İtiraz. Platform birden fazla ontolojiye izin vermektedir. Dolayısıyla “tek kano-nik anlam katmanı” tasviri yanlışır.

Değerlendirme. İtiraz teknik olarak doğrudur ve çalışma bu nedenle “karşılığ yok” iddiasını “farklı ölçekte” biçiminde revize etmiştir. Ancak itirazın gücü sınırlıdır: ontolojiler arasında bağ kurulamadığı için, çoklu ontoloji seçeneği ürünün temel değer önerisini fiilen iptal etmektedir. Sonuç, teknik olarak mümkün ancak ekonomik olarak cazibesiz bir seçenektir — ki bu, tasarımın yönlendirdiği davranışın tekil ontolojiye doğru olduğu anlamına gelir.

Dördüncü İtiraz: İdeal ile Pratiğin Kıyaslanması

İtiraz. Domain-Driven Design’in kendi literatürü, uygulamaların büyük kısmının DDD Lite olduğunu kabul etmektedir. Bu durumda karşılaştırma, gerçekleşmeyen bir ideal ile fiilen çalışan bir ürün arasında yapılmaktadır. Pratikte Ontology, pratikte uygulanan Domain-Driven Design’dan daha fazla dilsel disiplin üretiyor olabilir.

Değerlendirme. Bu, tezin karşılaştığı en ciddi itirazdır ve çalışma onu çürütememektedir. Verilebilecek tek yanıt şudur: Domain-Driven Design’in stratejik kısmı atlandığında bunun bir eksiklik olduğu literatürde adlandırılmış ve kayıt altına alınmıştır. Ontology’de aynı eksiklik için bir ad ve bir teşhis kriteri bulunmamaktadır. Yani fark, sonuçta değil, sonucun görünürlüğündedir — ki bu, çalışmanın merkezi tezinin kendisidir.

Buna karşın bu yanıt, karşılaştırmanın ampirik olarak sınanması gereğini ortadan kaldırmaz.

Beşinci İtiraz: Konfigürasyonun Yönetilebilirliği

İtiraz. Konfigürasyon değişikliklerinin izlenemez olduğu iddiası abartılıdır; platformlar sürüm kayıtları, yetkilendirme ve denetim izleri sunmaktadır.

Değerlendirme. İtiraz kısmen haklıdır. Çalışmanın iddiası izlenebilirliğin *teknik olarak imkânsız* olduğu değil, *sürtünmenin düşük* olduğudur. Kod değişikliği zorunlu bir inceleme sürecinden geçer ve tip sistemi tarafından denetlenir; konfigürasyon değişikliği için bu zorunluluk yapısal değil, kurumsal disipline bağlıdır. Fark niteliksel değil derecesel olmakla birlikte gerçektir.

Tezin Nihai Statüsü

Yukarıdaki itirazların ışığında, çalışmanın tezi şu daraltılmış biçimde savunulabilir:

Ontology, kavramların topluluğa göreliliğini kabul eden ancak bu görelilikten doğan müzakereyi kurumsal olarak adlandırmayan bir tasarımdır. Domain-Driven Design ise aynı müzakereyi adlandırır ve bir karar nesnesine dönüştürür. Bu fark, iki yaklaşımın etkinliğine ilişkin bir hüküm değil, hangi sorunun sorulabilir kaldığına ilişkin bir tespittir.

Sonuç

Her iki yaklaşım da aynı gözlemden hareket etmektedir: aynı kelime farklı topluluklarda farklı bir nesnedir ve bunu tek bir şemaya sıkıştırmak modeli bozar.

Cevapları ise ayırmaktadır.

Domain-Driven Design, sınırları çizmeyi, her sınırın içini dilsel olarak tutarlı tutmayı, sınırlar arasındaki güç ilişkisini açıkça adlandırmayı ve çeviri maliyetini kabul etmeyi önerir. Bedeli, sınırlar arası sorgulamanın pahalı olması ve koordinasyonun sürekli bir iş hâline gelmesidir.

Ontology, ortak bir nesne katmanı kurmayı, farklılığı bu katmanın üzerinde nesne tipleriyle ifade etmeyi ve tüm birimlerin aynı grafik üzerinde çalışmasını önerir. Bedeli, ortak katmanı tanımlayan tarafın fiili üstünlük kazanması ve bu üstünlüğün açık bir tasarım kararı olarak kaydedilmemesidir.

İkisi de aynı gerilimin iki ucudur: **anlamanın yerel olması ile sistemin bütünsel olarak sorgulanabilir olması arasındaki gerilim**. Bu gerilim çözülmüş değildir; her yaklaşım bir ucu seçmekte ve diğerinin bedelini ödemektedir.

Çalışmadan çıkan uygulamaya dönük sonuç şudur: bir kurumda ontoloji kuruluyorsa, Context Map'in yine de çizilmesi gerekmektedir. Haritadaki tüm oklar aynı yöne bakacaktır; belirleyici olan, bunun bilinçli bir tercih mi yoksa varsayılan bir sonuç mu olduğunun bilinmesidir.

Özet formülasyon:

Ontology, Domain-Driven Design'ın stratejik yarısını atlayıp taktiksel yarısını platform seviyesine çıkarma girişimidir. Kazandığı şey bağlantılanabilirlik, kaybettiği şey sınır müzakeresinin görünürlüğüdür.

İleri Araştırma Yönleri

Çalışmanın ampirik dayanaktan yoksun olması, üç somut araştırma sorusu doğurmaktadır.

1. Ontoloji kuran kurumlarda, nesne tipi tanımlarının hangi birim tarafından ve hangi müzakere süreciyle belirlendiğinin saha gözlemiyle incelenmesi.
2. Kanonik şemadan ontolojiye geçiş yapan kurumlarda, birimlerin kayıt dışı tablo tutma davranışının azalıp azalmadığının ölçülmesi.
3. Stratejik tasarımı uygulayan ve uygulamayan ekiplerde, kavramsal bulaıklığa bağlı hataların sıklığının karşılaştırılması.

Kaynakça

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (2003). "Anemic Domain Model". martinowler.com.

- Palantir Technologies. *Ontology Overview*. Foundry Dokümantasyonu.
- Palantir Technologies. *Ontologies Overview*. Foundry Dokümantasyonu.
- Ürkmez, Z. (2026). “DDD #1: Tasarım ve Yazılımın Sonunda Buluştuğu Yer”. Codewarts Bülten, 21 Ocak 2026.
- Ürkmez, Z. (2026). “DDD #2: Anlamların Sınırları”. Codewarts Bülten, 3 Şubat 2026.
- Ürkmez, Z. (2026). “DDD #3: Modellemeye Başlıyoruz, Bir E-Ticaret Domain’ini Sıfırdan Okumak”. Codewarts Bülten, 2 Mart 2026.
- Ürkmez, Z. (2026). “DDD #4: Context Map, Bounded Context’ler Arasındaki Güç Dengesi”. Codewarts Bülten, 21 Temmuz 2026.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
- Wittgenstein, L. (1921/2006). *Tractatus Logico-Philosophicus*. Çev. O. Aruoba. Metis Yayınları.
- Wittgenstein, L. (1953/2007). *Felsefi Soruşturmalar*. Çev. H. Barışcan. Metis Yayınları.